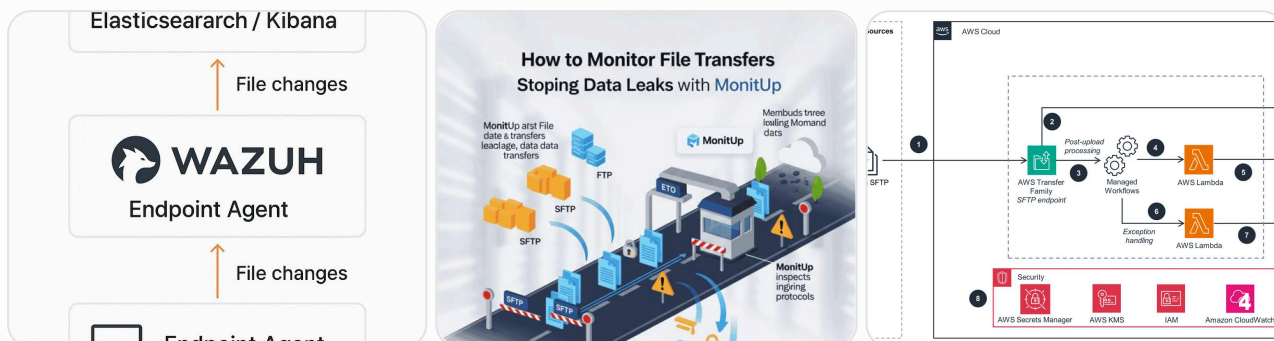
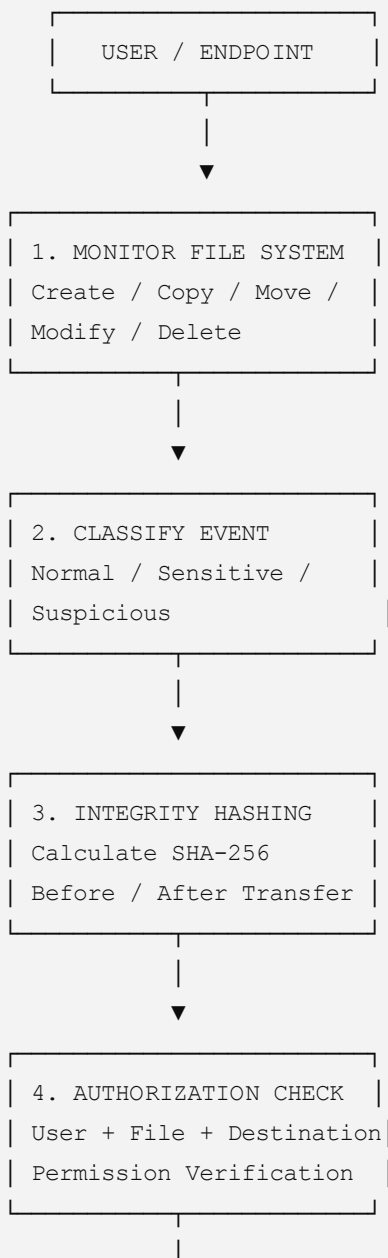


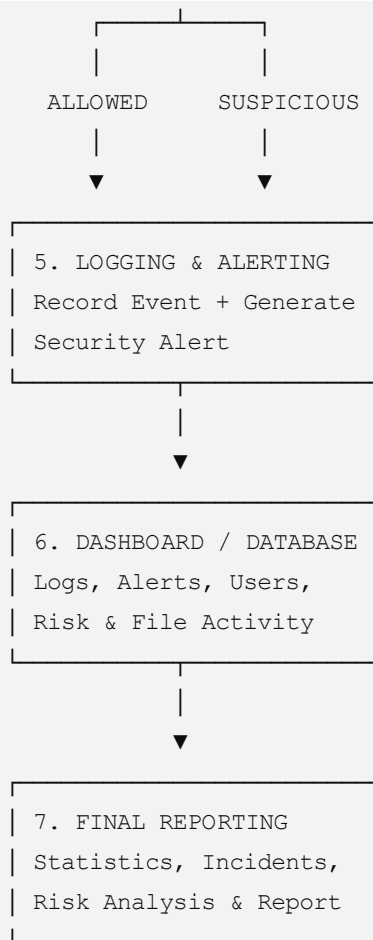
Secure File Transfer Monitoring System — Complete Workflow Architecture

The workflow of the **Secure File Transfer Monitoring System** consists of seven major stages: **File System Monitoring, Event Classification, Integrity Hashing, Authorization Check, Logging & Alerting, Dashboard Monitoring, and Final Reporting.**



Overall Workflow





1. Monitor File System

The first stage continuously monitors the selected file system, folders, or endpoints.

The monitoring engine detects:

- File creation
- File copying
- File movement
- File modification
- File deletion
- File transfer between monitored locations

For a Python implementation, **Watchdog** can be used to detect file-system events.

Example:

```
User copies:
confidential.pdf

From:
C:\Company\Documents
```

To:
E:\USB

Detected Event:
COPY

The system collects information such as:

Event ID
Username
File name
Source location
Destination location
Operation
File size
Timestamp

2. Classify Event

After detecting an event, the system determines the nature and risk of the file activity.

File classification

Normal

- |
- |— Documents
- |— Images
- |— General files

Sensitive

- |
- |— Financial documents
- |— Employee information
- |— Confidential reports

High Risk

- |
- |— Executable files
- |— Restricted information
- |— Unauthorized transfers

The system can also classify the **event itself**:

Normal Event
Suspicious Event
Unauthorized Event
High-Risk Event

For example:

```
confidential.xlsx
    ↓
Sensitive File
    ↓
Destination = USB
    ↓
Potentially Suspicious
```

3. Integrity Hashing

The third stage verifies file integrity using a cryptographic hash.

SHA-256 can be used for this purpose.

Before transfer

```
Original File
    ↓
SHA-256
    ↓
Original Hash
```

After transfer

```
Transferred File
    ↓
SHA-256
    ↓
New Hash
```

The system compares both values:

```
Original Hash == New Hash
    |
    +---+
    |   |
    YES NO
    |   |
    ▼   ▼
Integrity Possible
Maintained Modification
```

A hash mismatch can generate a high-priority security event.

4. Authorization Check

The system determines whether the user is authorized to perform the transfer.

The authorization decision can consider:

- User identity
- User role
- File classification
- Source location
- Destination location
- Transfer type
- Security policy

Example

```
User: employee01
File: confidential.xlsx
Destination: USB

Employee permission:
USB transfer = DENIED

Result:
UNAUTHORIZED TRANSFER
```

Another example:

```
User: security_admin
File: security_report.pdf
Destination: Approved Server

Permission:
ALLOWED

Result:
AUTHORIZED TRANSFER
```

5. Logging and Alerting

Every important event is recorded in the central database.

Example log

```
Event ID      : EVT-1025
User          : employee01
File          : confidential.xlsx
Operation     : COPY
Source        : C:\Company
Destination   : E:\USB
Classification : Sensitive
Authorization : DENIED
Risk          : HIGH
Timestamp     : 15-09-2026 14:05:22
```

Alert generation

If the event violates a security policy, the system generates an alert.

SECURITY ALERT	
Unauthorized File Transfer Detected	
User:	employee01
File:	confidential.xlsx
Destination:	USB
Risk:	HIGH

Alerts can be displayed through:

- Web dashboard
- Email
- SIEM
- Webhook
- System notification

6. Dashboard and Database

The collected events and alerts are stored in a database such as **SQLite** for a student/lab implementation.

The Flask dashboard can display:

SECURE FILE TRANSFER MONITOR	
Total Events	1,250
Allowed	1,100

Unauthorized	150	
Security Alerts	42	
Recent Events		
user01	report.pdf	LOW ALLOWED
user02	data.xlsx	HIGH BLOCKED
user03	test.exe	HIGH ALERT

The administrator can search and filter:

- Username
- File name
- Date/time
- Operation
- Risk level
- Authorization status
- Event type

7. Final Reporting

The final stage converts the collected monitoring information into a **security report**.

The report can contain:

File Transfer Statistics

```
Total Transfers      : 1,250
Authorized Transfers  : 1,100
Unauthorized Transfers: 150
```

Risk Statistics

```
Low Risk      : 900
Medium Risk   : 250
High Risk     : 80
Critical Risk  : 20
```

Incident Summary

```
Incident ID: INC-001
```

```
Type      : Unauthorized File Transfer
User      : employee01
```

File : confidential.xlsx
Destination: External USB
Severity : HIGH
Action : Blocked / Investigated

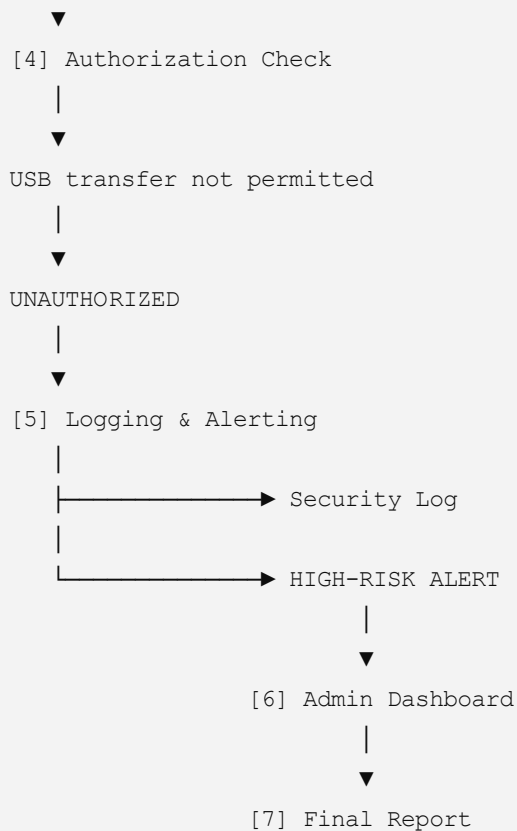
Final report sections

1. Executive Summary
2. Total File Transfers
3. Authorized vs Unauthorized Transfers
4. Sensitive File Activity
5. Integrity Verification Results
6. Security Alerts
7. High-Risk Incidents
8. User Activity
9. Risk Analysis
10. Actions Taken
11. Recommendations
12. Final Security Assessment

Complete End-to-End Example

Consider an employee attempting to copy a confidential file to an unauthorized USB drive.

```
Employee
|
▼
Copies confidential.xlsx
|
▼
[1] Monitor File System
|
▼
COPY event detected
|
▼
[2] Classify Event
|
▼
Sensitive File
|
▼
[3] Integrity Hashing
|
▼
SHA-256 calculated
|
```



Final Architecture Statement

The **Secure File Transfer Monitoring System** follows the workflow:

Monitor File System → Classify Event → Integrity Hashing → Authorization Check → Logging & Alerting → Dashboard/Database → Final Reporting

This architecture provides a complete monitoring cycle from **file activity detection to security reporting**. It is particularly suitable for a cybersecurity internship because it demonstrates **file-system monitoring, access control, cryptographic integrity verification, security logging, threat detection, alert management, and incident reporting**.